Otulia Web: Deep Dive - Chapter 5: Frontend Architecture (React & Vite)

5.1 Introduction to the Frontend

The Otulia frontend is a high-performance **Single Page Application (SPA)** built with **React 19** and bundled with **Vite**. It is designed for speed, modularity, and a premium "White Glove" user experience. By offloading rendering to the user's browser, we provide a smooth, app-like feel without traditional page reloads.

5.2 Global State & Context Providers

We avoid complex state libraries like Redux in favor of the **React Context API**, which provides a clean, native way to share data across the entire application.

5.2.1 The Authentication Provider (`AuthContext.jsx`)

This is the "Brain" of the frontend. It manages the user's session and provides security functions to all components.

```
export const AuthProvider = ({ children }) => {
    const [user, setUser] = useState(null); // Full user profile
    const [token, setToken] = useState(localStorage.getItem('token')); // JWT Passport

    // 1. Session Recovery: Runs automatically when the browser opens the site
    useEffect(() => {
        if (token) fetchUser(token); // Verify token with backend
    }, [token]);

    // 2. The Login Action: Accessible by any button in the app
    const login = async (email, password) => {
        const res = await fetch('/api/auth/login', { ... });
        if (res.ok) {
            const data = await res.json();
            localStorage.setItem('token', data.token); // Keep logged in on refresh
            setUser(data.user);
        }
    };

    return (
        <AuthContext.Provider value={{ user, token, login, logout }}>
            {children} {/* Every page inside the app can now see these values */}
        </AuthContext.Provider>
    );
};
```

5.3 Dynamic Routing (`App.jsx`)

We use `react-router-dom` to handle navigation. The `App.jsx` file acts as the primary "Switchboard" for the platform.

5.3.1 Route Mapping & Layout Control

```
function App() {
  const location = useLocation();

  // 1. Intelligent UI: Hide the footer on Admin and Auth pages to maintain focus
  const hideFooterRoutes = ['/admin', '/login', '/signup', '/inventory'];
  const shouldShowFooter = !hideFooterRoutes.some(path =>
location.pathname.startsWith(path));

  return (
    <CartProvider>
      <ScrollToTop /> {/* Forces the browser to start at the top on every link click */}
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/shop" element={<Shop />} />
        {/* 2. Dynamic Asset Routing: handles any category (car, yacht, etc.) */}
        <Route path="/asset/:category/:id" element={<Asset />} />

        {/* 3. Protected Routes: Private pages like Profile */}
        <Route path="/profile" element={<Profile />} />
      </Routes>
      {shouldShowFooter && <Footer />}
    </CartProvider>
  )
}
```

5.4 The Component-Driven Design System

Otulia's UI is built using reusable building blocks. This ensures that a car card looks and behaves exactly like a yacht card.

5.4.1 The Smart Asset Card (`AssetCard.jsx`)

The `AssetCard` is more than just a display; it's a mini-application that handles its own favorites and hover effects.

```
const AssetCard = ({ item }) => {
  // 1. Interactive Logic: Local state for UI feedback
  const [isLiked, setIsLiked] = useState(false);
  const [isHovered, setIsHovered] = useState(false);

  // 2. Category Inference: Automatically detects if it should show car specs or house specs
  let displayDetails = item.keySpecifications?.power || item.keySpecifications?.bedrooms;

  // 3. Dynamic Navigation
  const handleClick = () => navigate(`/asset/${item.category}/${item._id}`);

  return (
    <div onMouseEnter={() => setIsHovered(true)} onClick={handleClick}>
      {/* 4. Optimistic UI: The heart changes color instantly when clicked */}
```

```
      <button onClick={handleHeartToggle}>
        <HeartIcon color={isLiked ? "red" : "white"} />
      </button>

      {/* 5. Luxury Polish: Images scale up smoothly on hover */}
      <img className={`transition-transform duration-1000 ${isHovered ? "scale-110" : ""}`}
/>
    </div>
  );
};
```

5.5 High-End Styling with Tailwind CSS 4

The luxury look is achieved through utility-first CSS.

- **Responsiveness**: We use prefixes like `md:` and `lg:` to ensure the site looks perfect on an iPhone, an iPad, and a 4K monitor.
- **Typography**:
  - `font-playfair` : Used for headers to evoke heritage and status.
  - `font-montserrat` : Used for data and UI elements for modern readability.
- **Glassmorphism**: We use `backdrop-blur-md` and `bg-white/80` to create sophisticated, semi-transparent overlays common in luxury interfaces.

5.6 Performance Optimizations

- **Code Splitting**: Vite automatically breaks our code into small "chunks." Instead of downloading the whole website, the user only downloads the code for the page they are currently visiting.
- **Image Handling**: We never load full-size images in the `AssetCard` . We request "optimized thumbnails" from the Cloudinary API to ensure fast scroll speeds.
- **Vite Proxy**: In development, we use a proxy in `vite.config.js` so that the frontend can talk to the backend without hitting "CORS" errors on your local machine.